TP n°4 : Introduction au Procedural Language PL/SQL d'Oracle – Partie I

Salaheddine Aitouahmane — 13 avril 2026

Exercice 1 – Structures de base en PL/SQL

1.1 Somme de deux entiers

Concevoir un bloc anonyme PL/SQL invitant l'utilisateur à fournir deux entiers, puis affichant leur somme.

```
SET SERVEROUTPUT ON;
DECLARE
    v_a NUMBER;
    v_b NUMBER;
    v_somme NUMBER;
BEGIN
    v_a := &entier1;
    v_b := &entier2;
    v_somme := v_a + v_b;
    DBMS_OUTPUT.PUT_LINE('La somme de ' || v_a || ' et '
        || v_b || ' est : ' || v_somme);
END;
/
```

Analyse. La syntaxe `&variable` correspond aux variables de substitution d'Oracle : SQL Developer ouvre une boîte de dialogue demandant la valeur à l'utilisateur lors de l'exécution. Le résultat est ensuite affiché dans le tampon de sortie via `DBMS_OUTPUT.PUT_LINE`.

1.2 Table de multiplication

Concevoir un bloc anonyme lisant un nombre auprès de l'utilisateur, puis affichant l'intégralité de sa table de multiplication (de 1 à 10).

```
SET SERVEROUTPUT ON;
DECLARE
    v_nombre NUMBER;
BEGIN
    v_nombre := &nombre;
    DBMS_OUTPUT.PUT_LINE('Table de multiplication de ' ||
       v_nombre || ' :');
    FOR i IN 1..10 LOOP
```

```
        DBMS_OUTPUT.PUT_LINE(v_nombre || ' x ' || i || ' = '
            || (v_nombre * i));
    END LOOP;
END;
/
```

Analyse. La boucle `FOR i IN 1..10` parcourt automatiquement les valeurs de 1 à 10, sans gestion manuelle de compteur. L'opérateur `||` concatène les différentes parties de la chaîne affichée à chaque itération.

1.3 Puissance récursive x^n

Implémenter une fonction récursive retournant x^n, où x et n sont deux entiers positifs.

```
SET SERVEROUTPUT ON;
-- Création de la fonction récursive
CREATE OR REPLACE FUNCTION puissance(x NUMBER, n NUMBER)
RETURN NUMBER
IS
BEGIN
    IF n = 0 THEN
        RETURN 1;
    ELSE
        RETURN x * puissance(x, n - 1);
    END IF;
END;
/

-- Test de la fonction
DECLARE
    v_x NUMBER := &x;
    v_n NUMBER := &n;
    v_resultat NUMBER;
BEGIN
    v_resultat := puissance(v_x, v_n);
    DBMS_OUTPUT.PUT_LINE(v_x || '^' || v_n || ' = ' ||
       v_resultat);
END;
/
```

Analyse. Le cas d'arrêt correspond à $n = 0$, pour lequel la fonction renvoie 1 (puisque $x^0 = 1$). Dans tout autre cas, elle se rappelle avec un exposant diminué de un : $x^n = x \times x^{n-1}$. L'instruction `CREATE OR REPLACE FUNCTION` rend la fonction persistante dans le schéma de la base.

1.4 Factorielle avec stockage en table

Concevoir un bloc anonyme calculant la factorielle d'un entier strictement positif saisi par l'utilisateur. Le résultat doit être persisté dans une table `resultatFactoriel`*.*

```
-- Création de la table de résultats
```

```
CREATE TABLE resultatFactoriel (
    nombre      NUMBER(10) NOT NULL,
    factorielle NUMBER(38) NOT NULL,
    CONSTRAINT pk_fact PRIMARY KEY (nombre)
);

-- Bloc anonyme de calcul
SET SERVEROUTPUT ON;
DECLARE
    v_nombre NUMBER;
    v_fact   NUMBER := 1;
BEGIN
    v_nombre := &nombre;

    IF v_nombre <= 0 THEN
        DBMS_OUTPUT.PUT_LINE('Erreur : le nombre doit etre'
            || ' strictement positif.');
    ELSE
        -- Calcul de la factorielle
        FOR i IN 1..v_nombre LOOP
            v_fact := v_fact * i;
        END LOOP;

        -- Stockage du résultat dans la table
        INSERT INTO resultatFactoriel (nombre, factorielle)
        VALUES (v_nombre, v_fact);
        COMMIT;

        DBMS_OUTPUT.PUT_LINE(v_nombre || '! = ' || v_fact);
    END IF;
END;
/
```

Analyse. La table `resultatFactoriel` possède une clé primaire sur `nombre`, garantissant l'unicité de chaque entrée. Le calcul itératif accumule le produit dans `v_fact` via la boucle `FOR`. L'instruction `COMMIT` rend l'insertion définitive.

1.5 Factorielles des 20 premiers entiers

Adapter le programme précédent pour calculer et stocker dans une table `resultatsFactoriels` *les factorielles de tous les entiers de 1 à 20.*

```
-- Création de la table
CREATE TABLE resultatsFactoriels (
    nombre      NUMBER(10) NOT NULL,
    factorielle NUMBER(38) NOT NULL,
    CONSTRAINT pk_facts PRIMARY KEY (nombre)
);

-- Bloc anonyme de calcul et stockage
SET SERVEROUTPUT ON;
```

```
DECLARE
    v_fact NUMBER;
BEGIN
    FOR n IN 1..20 LOOP
        v_fact := 1;
        FOR i IN 1..n LOOP
            v_fact := v_fact * i;
        END LOOP;

        INSERT INTO resultatsFactoriels (nombre, factorielle)
        VALUES (n, v_fact);

        DBMS_OUTPUT.PUT_LINE(n || '! = ' || v_fact);
    END LOOP;
    COMMIT;
END;
/

-- Vérification
SELECT * FROM resultatsFactoriels ORDER BY nombre;
```

Analyse. Deux boucles imbriquées : la boucle externe génère chaque entier n de 1 à 20, la boucle interne calcule $n!$. La variable `v_fact` est réinitialisée à 1 au début de chaque itération externe. Un unique `COMMIT` après la boucle valide l'ensemble des 20 insertions en une seule transaction.

Exercice 2 – Manipulation de la table emp

Préparation de l'environnement : création de la table et alimentation avec un jeu de données initial.

```
CREATE TABLE emp (
    matr    NUMBER(10)   NOT NULL,
    nom     VARCHAR2(50) NOT NULL,
    sal     NUMBER(7,2),
    adresse VARCHAR2(96),
    dep     NUMBER(10)   NOT NULL,
    CONSTRAINT emp_pk PRIMARY KEY (matr)
);

-- Insertion de données de test
INSERT INTO emp VALUES (1, 'Ali', 3000,
    'rue de Paris', 92000);
INSERT INTO emp VALUES (2, 'Sara', 3500,
    'avenue Victor Hugo', 75000);
INSERT INTO emp VALUES (3, 'Mohamed', 2800,
    'boulevard Haussmann', 75000);
INSERT INTO emp VALUES (5, 'Fatima', 4000,
    'rue de la Paix', 92000);
INSERT INTO emp VALUES (6, 'Karim', 2200,
    'rue du Commerce', 10);
INSERT INTO emp VALUES (7, 'Nadia', 3100,
    'place de la Nation', 10);
COMMIT;
```

2.1 Insertion via %ROWTYPE

Rédiger un bloc anonyme insérant un nouvel employé dont les attributs ***matr****,* ***nom****,* ***sal****,* ***adresse*** *et* ***dep*** *valent respectivement 4, Youcef, 2500, avenue de la République, 92002.*

```
SET SERVEROUTPUT ON;
DECLARE
    v_employe emp%ROWTYPE;
BEGIN
    v_employe.matr    := 4;
    v_employe.nom     := 'Youcef';
    v_employe.sal     := 2500;
    v_employe.adresse := 'avenue de la Republique';
    v_employe.dep     := 92002;

    -- Insertion du tuple dans la table
    INSERT INTO emp VALUES v_employe;
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Employe insere avec succes.');
END;
/
```

Analyse. L'attribut `%ROWTYPE` crée une variable structurée miroir de la table `emp` : chaque champ de l'enregistrement correspond à une colonne. L'affectation se fait champ par champ via l'opérateur `:=` et la notation pointée. L'insertion de l'enregistrement complet se fait directement par `INSERT INTO emp VALUES v_employe`, sans lister les colonnes.

2.2 Suppression et `SQL%ROWCOUNT`

Rédiger un bloc anonyme supprimant tous les employés dont le département vaut 10, puis affichant le nombre de lignes effectivement supprimées grâce à `SQL%ROWCOUNT`.

```
1  SET SERVEROUTPUT ON;
2  DECLARE
3      v_nb_lignes NUMBER;
4  BEGIN
5      DELETE FROM emp WHERE dep = 10;
6      v_nb_lignes := SQL%ROWCOUNT;
7
8      DBMS_OUTPUT.PUT_LINE('Nombre de lignes supprimees : '
9          || v_nb_lignes);
10     COMMIT;
11 END;
12 /
```

Analyse. `SQL%ROWCOUNT` est un attribut du curseur implicite mis à jour automatiquement après chaque opération DML. Il renvoie le nombre exact de lignes touchées par la dernière instruction exécutée, ce qui permet de contrôler l'impact de la suppression.

2.3 Somme des salaires – curseur explicite et `LOOP`

Rédiger un bloc anonyme affichant la somme de tous les salaires à l'aide d'un curseur explicite et d'une boucle `LOOP`. Le résultat doit être identique à : `SELECT SUM(sal) FROM emp`.

```
1  SET SERVEROUTPUT ON;
2  DECLARE
3      v_salaire EMP.sal%TYPE;
4      v_total   EMP.sal%TYPE := 0;
5      CURSOR c_salaires IS
6          SELECT sal FROM emp;
7  BEGIN
8      OPEN c_salaires;
9
10     LOOP
11         FETCH c_salaires INTO v_salaire;
12         EXIT WHEN c_salaires%NOTFOUND;
13         IF v_salaire IS NOT NULL THEN
14             v_total := v_total + v_salaire;
15         END IF;
16     END LOOP;
17
18     CLOSE c_salaires;
```

```
    DBMS_OUTPUT.PUT_LINE('Somme des salaires : ' || v_total);
END;
/
```

Analyse. Un curseur explicite nomme une requête pour la parcourir ligne par ligne. Le cycle de vie est géré manuellement : `OPEN` initialise l'exécution, `FETCH` récupère la ligne courante, et `CLOSE` libère les ressources. L'attribut `%NOTFOUND` passe à `TRUE` dès que le curseur a épuisé toutes les lignes, déclenchant la sortie de boucle. Le test `IS NOT NULL` protège contre les salaires non renseignés.

2.4 Salaire moyen – curseur explicite et `LOOP`

Adapter la procédure précédente pour calculer et afficher le salaire moyen. Le résultat doit correspondre à : `SELECT AVG(sal) FROM emp`.

```
SET SERVEROUTPUT ON;
DECLARE
    v_salaire  EMP.sal%TYPE;
    v_total    EMP.sal%TYPE := 0;
    v_compteur NUMBER := 0;
    v_moyenne  NUMBER(10,2);
    CURSOR c_salaires IS
        SELECT sal FROM emp;
BEGIN
    OPEN c_salaires;

    LOOP
        FETCH c_salaires INTO v_salaire;
        EXIT WHEN c_salaires%NOTFOUND;
        IF v_salaire IS NOT NULL THEN
            v_total := v_total + v_salaire;
            v_compteur := v_compteur + 1;
        END IF;
    END LOOP;

    CLOSE c_salaires;

    IF v_compteur > 0 THEN
        v_moyenne := v_total / v_compteur;
        DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || v_moyenne);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Aucun salaire trouve.');
    END IF;
END;
/
```

Analyse. Un compteur `v_compteur` est incrémenté à chaque salaire valide rencontré. La moyenne s'obtient par le rapport `v_total / v_compteur`. Un garde-fou vérifie que le compteur est non nul avant la division, prévenant toute erreur arithmétique.

2.5 Somme et moyenne – boucle `FOR IN`

Réécrire les deux procédures précédentes en remplaçant la boucle `LOOP/FETCH` par la syntaxe simplifiée `FOR ...IN`.

a) Somme des salaires :

```
1  SET SERVEROUTPUT ON;
2  DECLARE
3      v_total EMP.sal%TYPE := 0;
4      CURSOR c_salaires IS
5          SELECT sal FROM emp;
6  BEGIN
7      FOR v_rec IN c_salaires LOOP
8          IF v_rec.sal IS NOT NULL THEN
9              v_total := v_total + v_rec.sal;
10         END IF;
11     END LOOP;
12
13     DBMS_OUTPUT.PUT_LINE('Somme des salaires : ' || v_total);
14 END;
15 /
```

b) Salaire moyen :

```
1  SET SERVEROUTPUT ON;
2  DECLARE
3      v_total    EMP.sal%TYPE := 0;
4      v_compteur NUMBER := 0;
5      v_moyenne  NUMBER(10,2);
6      CURSOR c_salaires IS
7          SELECT sal FROM emp;
8  BEGIN
9      FOR v_rec IN c_salaires LOOP
10         IF v_rec.sal IS NOT NULL THEN
11             v_total := v_total + v_rec.sal;
12             v_compteur := v_compteur + 1;
13         END IF;
14     END LOOP;
15
16     IF v_compteur > 0 THEN
17         v_moyenne := v_total / v_compteur;
18         DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || v_moyenne);
19     ELSE
20         DBMS_OUTPUT.PUT_LINE('Aucun salaire trouve.');
21     END IF;
22 END;
23 /
```

Analyse. La construction `FOR v_rec IN c_salaires LOOP` prend en charge l'intégralité du cycle de vie du curseur : `OPEN`, `FETCH`, test de fin et `CLOSE` sont implicites. La variable

`v_rec` est un enregistrement déclaré automatiquement par le compilateur, dont les champs reflètent les colonnes du `SELECT`. Cette écriture est plus concise et moins sujette aux oublis.

2.6 Curseur paramétré

Rédiger un bloc anonyme affichant les noms des employés des départements 92000 et 75000, en s'appuyant sur un curseur paramétré.

```
SET SERVEROUTPUT ON;
DECLARE
    CURSOR c(p_dep EMP.dep%TYPE) IS
        SELECT dep, nom
        FROM emp
        WHERE dep = p_dep;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- Departement 92000 ---');
    FOR v_employe IN c(92000) LOOP
        DBMS_OUTPUT.PUT_LINE('  ' || v_employe.nom);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('--- Departement 75000 ---');
    FOR v_employe IN c(75000) LOOP
        DBMS_OUTPUT.PUT_LINE('  ' || v_employe.nom);
    END LOOP;
END;
/
```

Analyse. Le curseur `c` reçoit un paramètre formel `p_dep` typé par `EMP.dep%TYPE`, garantissant la cohérence de type avec la colonne cible. Ce mécanisme de paramétrage permet de réutiliser la même définition de curseur pour filtrer des départements différents, en passant simplement une valeur réelle lors de l'appel : `c(92000)` puis `c(75000)`.

Exercice 3 – Package avec surcharge et exceptions

Implémenter la spécification du package de gestion des clients, composé de deux procédures homonymes `ajouter_client` *(surcharge). Le corps du package doit lever et traiter toutes les exceptions pertinentes.*

3.1 Table `client`

```
CREATE TABLE client (
    id_client NUMBER(10)   NOT NULL,
    nom       VARCHAR2(50) NOT NULL,
    prenom    VARCHAR2(50),
    adresse   VARCHAR2(100),
    telephone VARCHAR2(20),
    CONSTRAINT client_pk PRIMARY KEY (id_client)
);
```

3.2 Spécification du package

```
CREATE OR REPLACE PACKAGE gestion_client AS
    -- Procédure 1 : ajout avec tous les attributs
    PROCEDURE ajouter_client(
        p_id        IN client.id_client%TYPE,
        p_nom       IN client.nom%TYPE,
        p_prenom    IN client.prenom%TYPE,
        p_adresse   IN client.adresse%TYPE,
        p_telephone IN client.telephone%TYPE
    );

    -- Procédure 2 : ajout avec seulement id et nom
    -- (surcharge : même nom, signature différente)
    PROCEDURE ajouter_client(
        p_id  IN client.id_client%TYPE,
        p_nom IN client.nom%TYPE
    );
END gestion_client;
/
```

3.3 Corps du package

```
CREATE OR REPLACE PACKAGE BODY gestion_client AS

    -- Procédure 1 : ajout complet
    PROCEDURE ajouter_client(
        p_id        IN client.id_client%TYPE,
        p_nom       IN client.nom%TYPE,
        p_prenom    IN client.prenom%TYPE,
        p_adresse   IN client.adresse%TYPE,
        p_telephone IN client.telephone%TYPE
    ) IS
```

```
    BEGIN
        INSERT INTO client (id_client, nom, prenom,
            adresse, telephone)
        VALUES (p_id, p_nom, p_prenom,
            p_adresse, p_telephone);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Client ' || p_nom
            || ' ajoute avec succes (complet).');

    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            DBMS_OUTPUT.PUT_LINE('Erreur : le client avec id '
                || p_id || ' existe deja.');
            ROLLBACK;
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Erreur inattendue : '
                || SQLERRM);
            ROLLBACK;
    END ajouter_client;

    -- Procédure 2 : ajout avec id et nom seulement
    PROCEDURE ajouter_client(
        p_id  IN client.id_client%TYPE,
        p_nom IN client.nom%TYPE
    ) IS
    BEGIN
        INSERT INTO client (id_client, nom, prenom,
            adresse, telephone)
        VALUES (p_id, p_nom, NULL, NULL, NULL);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Client ' || p_nom
            || ' ajoute avec succes (simplifie).');

    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            DBMS_OUTPUT.PUT_LINE('Erreur : le client avec id '
                || p_id || ' existe deja.');
            ROLLBACK;
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Erreur inattendue : '
                || SQLERRM);
            ROLLBACK;
    END ajouter_client;

END gestion_client;
/
```

3.4 Jeu de tests

```
SET SERVEROUTPUT ON;

```

```
-- Test 1 : Ajout complet
BEGIN
    gestion_client.ajouter_client(
        1, 'Dupont', 'Jean',
        '12 rue de la Paix', '0612345678');
END;
/

-- Test 2 : Ajout simplifié (surcharge)
BEGIN
    gestion_client.ajouter_client(2, 'Martin');
END;
/

-- Test 3 : Tentative de doublon (exception DUP_VAL_ON_INDEX)
BEGIN
    gestion_client.ajouter_client(1, 'Durand');
END;
/

-- Vérification
SELECT * FROM client;
```

Résultats attendus.

1. **Test 1** : insertion réussie avec les cinq attributs — message de confirmation.
2. **Test 2** : insertion réussie avec seulement l'identifiant et le nom (les champs optionnels restent à `NULL`) — message de confirmation.
3. **Test 3** : l'identifiant 1 existe déjà — l'exception `DUP_VAL_ON_INDEX` est interceptée, la transaction est annulée par `ROLLBACK`, et un message d'erreur explicite est affiché.

Analyse. La surcharge (*overloading*) permet de définir plusieurs procédures de même nom au sein d'un package, à condition que leurs signatures diffèrent. Le compilateur sélectionne la version adéquate en fonction des arguments fournis à l'appel. Côté exceptions : `DUP_VAL_ON_INDEX` est levée en cas de violation de contrainte d'unicité (clé primaire), tandis que `WHEN OTHERS` capture toute exception non explicitement traitée. `SQLERRM` retourne le libellé complet de l'erreur Oracle. Le `ROLLBACK` dans chaque gestionnaire annule les modifications partielles, préservant la cohérence de la base.